TITLE AND DATE

Document name: CMPE311 Project 3 (PROJECT-MYMILLI), Revision 3 of the CMPE311 Project 1 design document

Document reference: provost.cmpe311.fall26.project#3

Supersedes: provost.cmpe311.fall26.project#2 [2]

Change authority: ECO 0.3 [4]; notified by ECN 0.3 [3]

Date of publication: October 15, 2026

Lead engineer: Raheem Provost, UMBC

STAKEHOLDERS

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

REVISION HISTORY

Table 1. Revision history

Rev	Date	Author	Change
1	September 22, 2026	RP	Initial release (Project 1, PROJECT-ASYNC) [1]
2	October 3, 2026	RP	ECO 0.1 / ECN 0.1: task dispatch moved to a round-robin cyclic executive using a function pointer table; EC.6, T.7 added [2]
3	October 15, 2026	RP	ECO 0.3 / ECN 0.3: `millis()` replaced by `mymilli()` with its own Timer/Counter0 compare-match ISR and register constants (new files mymilli.h and mymilli.cpp); EC.7, EC.7.1, EC.7.2, T.8 to T.10 added; #include check of T.6 and T.7 changed

HIGH-LEVEL DESCRIPTION

This document is the project report for Project #3 of the CMPE311 class. It is Revision 3 of the Project #1 design document [1] and records the design change required by ECO 0.3 [4]. Sections that did not change are cited to [2] and [1] rather than restated.

DESCRIPTION

This design implements an embedded system on an Arduino Uno R3 development platform that independently controls the blink rate of two LEDs. The User interacts with the System through the Arduino IDE serial monitor, specifying an LED number and a blink interval in milliseconds. In this revision the System reads time from `mymilli()`, written by The Developer, instead of the Arduino `millis()`. The behavior seen by The User is identical to Project #2.

RESULT SUMMARY

The project was a success. The embedded system design replaced the Arduino millis() function with mymilli(), driven by its own Timer/Counter0 compare-match ISR and register constants, and met all testing (T.1 to T.10) and high-level requirements. The behavior seen by The User is identical to Project #2.

REFERENCES AND GLOSSARY

REFERENCES

1. provost.cmpe311.fall26.project#1, CMPE311 Project 1 design document (PROJECT-ASYNC), Revision 1, R. Provost, September 22, 2026.
2. provost.cmpe311.fall26.project#2, CMPE311 Project 2 design document (PROJECT-CE), Revision 2, R. Provost, October 3, 2026.
3. provost.cmpe311.fall26.project#3.ECN, Engineering Change Note ECN 0.3, R. Provost, October 15, 2026.
4. PROJECT#3: PROJECT-MYMILLI assignment, including Engineering Change Order ECO 0.3 (3/1/2030, requestor Prof. Kidd), CMPE311.
5. Microchip Technology, ATmega328P Data Sheet: 8-bit Timer/Counter0 with PWM, register summary and interrupt vector table.

DEFINITIONS

The definitions of The User, The System, The Customer, The Developer, The Evaluator, The Customer-requirements, The HL-requirements, The Requirements, The Educational-constraints, The Company, The Contract, serial-monitor and The Reference-platform are identical to those of PROJECT-ASYNC. See [1] REFERENCES AND GLOSSARY section, DEFINITIONS. The terms added in Revision 2 (cyclic executive, round-robin, task, task table, dispatch round) are unchanged; see [2]. The following are added in this revision:

"arduino.h" – Helper functions and constants added to the gcc environment by Arduino (maintained by Arduino AG, owned by Qualcomm).

"avr.h" – Helper functions and constants added to the gcc environment by AVR (Microchip), including the register names and bit names.

"millis()" – The original millisecond clock function supplied by arduino.h, which this revision replaces.

"mymilli()" – The replacement for `millis()` written by The Developer. It returns the number of milliseconds since the time base was started by `mymilliInit()`, as an unsigned long.

"mymilli.h", "mymilli.cpp" – The header and source files, written by The Developer, that hold the time base.

"Time base" – The Timer/Counter0 setup, the compare-match ISR and the counter that keep the millisecond count.

"Tick" – One compare match of Timer/Counter0, which occurs every 1,000 microseconds (1 ms) in this design.

ACRONYMS AND ABBREVIATIONS

The acronyms and abbreviations of [1] and [2] REFERENCES AND GLOSSARY sections are unchanged. The following are added in this revision:

CTC – Clear Timer on Compare Match

ISR – Interrupt Service Routine

PROJECT-MYMILLI – Project #3, documented in this document

PWM – Pulse Width Modulation

REQUIREMENTS

CONVENTIONS

The conventions are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, CONVENTIONS. Requirements added after Revision 2 are tagged with the date and the author's initials, e.g. "EC.7 (added 10/15/26 – RP)".

PART I – CUSTOMER REQUIREMENTS AND DERIVED HIGH-LEVEL REQUIREMENTS

ECO 0.3 makes no change to the customer requirements or the high-level requirements [4]. The customer requirements, C1 through C6, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART I, Table 1. The high-level technical requirements derived from them, HL.1 through HL.6 of Part I, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART I, Table 2.

PART II – HIGH-LEVEL REQUIREMENTS AND EDUCATIONAL CONSTRAINTS

The high-level technical requirements, HL.1 through HL.6, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART II, Table 3.

The educational constraints, EC.1 through EC.5 including EC.2.1, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART II, Table 4. EC.6 is unchanged from PROJECT-CE. See [2] REQUIREMENTS section, Table 2. The following educational constraints are added to implement ECO 0.3 [4]:

Table 2. Educational constraints added in this revision

ID	Educational constraint	Source
EC.7 (added 10/15/26 – RP)	The Developer must replace the `millis()` function provided by arduino.h with a function written by The Developer and named `mymilli()`. The replacement may extend the capability of `millis()`.	ECO 0.3, Educational Constraint 1.0 [4]
EC.7.1 (added 10/15/26 – RP)	The Developer must write their own PWM counter ISR and use it as the timing mechanism underlying `mymilli()`.	ECO 0.3, Educational Constraint 1.0.1 [4]
EC.7.2 (added 10/15/26 – RP)	The Developer must define their own register constants that would normally be defined in avr.h. The constants need not have the same names as those in avr.h.	ECO 0.3, Educational Constraint 1.0.2 [4]

Informative: EC.7 to EC.7.2 constrain only the source of time. They do not relax EC.5 or EC.6; the time base is bare-metal code written by The Developer. EC.7.1 asks for a PWM counter ISR: the time base uses Timer/Counter0, the ATmega328P counter that also generates PWM, in clear-timer-on-compare-match (CTC) mode rather than in a PWM mode, so that its period is exactly 1 ms. The remaining items of ECO 0.3 (1.1, 1.1.1, 1.2 and 1.3) are requirements on the testing, the report and the notification, and are met as shown in Table 3.

Table 3. How each item of ECO 0.3 is met

ECO item	Requirement	Where it is met
1.0	Replace `millis()` with `mymilli()`	EC.7; SOFTWARE DESIGN; Appendix B
1.0.1	Own PWM counter ISR as the timing mechanism	EC.7.1; SOFTWARE DESIGN, Compare-match ISR
1.0.2	Own register constants	EC.7.2; SOFTWARE DESIGN, Table 5
1.1	Testing and validation requirements include those of PROJECT-CE	T.1 to T.7 re-run on this build (Table 9)
1.1.1	Extend the tests to evaluate the modified system	T.8 added (Table 6)
1.2	New revision of the formal design document, noting design changes	This document (Revision 3); changes listed in ECN 0.3 [3]
1.3	Properly formatted ECN notifying other teams	ECN 0.3 [3]

DESIGN

DESIGN PRE-REQUISITES

The design pre-requisites are identical to those of PROJECT-ASYNC. See [1] DESIGN section, DESIGN PRE-REQUISITES.

BILL OF MATERIALS

The bill of materials is identical to that of PROJECT-ASYNC. See [1] DESIGN section, Table 5.

DEVELOPMENT PLATFORM

The development platform is identical to that of PROJECT-ASYNC: Arduino IDE 2.3.10 on Windows 11, with the AVR-GCC toolchain bundled in the Arduino AVR Boards package 1.8.7 (EC.1). See [1] DESIGN section, DEVELOPMENT PLATFORM.

SYSTEM OVERVIEW (CHANGED)

The User still interacts with the System entirely through the Arduino IDE serial monitor, with the same prompts as PROJECT-ASYNC. The work is still split into the three tasks of PROJECT-CE (`askQuestions()`, `serviceLED1()`, `serviceLED2()`), dispatched one per pass of `loop()` from `taskTable[]`. See [2] DESIGN section, SYSTEM OVERVIEW and Table 3.

What changed is where the System gets its sense of time. In PROJECT-CE the two LED tasks read `millis()`, whose count is kept by a timer ISR inside the Arduino core. In this revision they read `mymilli()`. Its count is kept by an ISR written by The Developer, and the timer and interrupt registers it uses are set through register constants defined in `mymilli.cpp`. Figure 1 shows the path from the clock to the blink tasks.

HARDWARE DESIGN

The hardware design is identical to that of PROJECT-ASYNC: LED1 on digital pin 2 and LED2 on digital pin 3, each through a 330 Ω current-limiting resistor to a common ground. See [1] DESIGN section, HARDWARE DESIGN, Figure 1 (Schematic) and Table 6 (Arduino pin assignments). There is no wiring change. Timer/Counter0 is a peripheral inside the ATmega328P; none of its output pins (digital pins 5 and 6) is connected to the System.

SOFTWARE DESIGN (CHANGED)

The program keeps the PROJECT-CE task table, dispatcher, serial dialog and blink logic. See [2] DESIGN section, SOFTWARE DESIGN, Table 3. The changes are the new time base, which lives in two new files, `mymilli.h` and `mymilli.cpp`; one added line at the top of the sketch, `#include "mymilli.h"`; one added call, `mymilliInit()`, at the end of `setup()`; and the replacement of `millis()` by `mymilli()` inside `blinkLED1()` and `blinkLED2()`.

Timer selection. The time base uses Timer/Counter0, an 8-bit timer. With the clock divided by 64 it counts every 4 µs, so a compare value of 249 gives a match every 250 counts, which is exactly 1,000 µs, and no correction is needed. The Arduino core also uses Timer/Counter0 for its own `millis()`, but only through its overflow interrupt, so the compare-match A interrupt vector used here is free and there is no duplicate-vector link error. The cost is that the core's `millis()`, `micros()` and `delay()` stop working once `mymilliInit()` has reprogrammed the timer (see Software Design, Arduino timing functions). Timer/Counter2 is not used because its output compare pin OC2B is digital pin 3, which is LED2.

Timer configuration. Timer/Counter0 is set to CTC mode (waveform generation bits WGM01:0 = 0b10), with the compare value in OCR0A set to 249 (0b11111001) and the clock divided by 64 (clock select bits

0b011). At 16 MHz the timer counts every 4 µs and returns to zero after counting from 0 to 249, which is 250 counts, so a compare match occurs every 1,000 µs. The compare-match A interrupt is enabled, and that interrupt is the ISR required by EC.7.1. Table 4 gives the parameters.

Table 4. Time base parameters

Parameter	Value	Basis
System clock	16,000,000 Hz	Reference platform
Timer/Counter0 mode	CTC (WGM01:0 = 0b10)	Counter returns to zero on a match with OCR0A [5]
Clock select	System clock / 64 (0b011)	250 kHz, 4 µs per count
Compare value (OCR0A)	249 (0b11111001)	Counts 0 to 249
Counts per tick	250	OCR0A + 1
Tick period	1,000 µs	250 × 4 µs
Added per tick	1 ms	One increment of the count; no correction needed
Interrupt rate	1,000 per second	1 ÷ 1 ms
Count width and range	32-bit unsigned; wraps after about 49.7 days	2^32 ms

Compare-match ISR. The ISR is declared with the AVR `ISR()` macro for the `TIMER0_COMPA_vect` vector. It runs once per tick and adds 1 to the 32-bit count `millicount`. Because the tick is exactly 1 ms, the count equals the elapsed milliseconds and advances by exactly 1 each time; it skips a value only if an interrupt is missed, which would need interrupts to be off for more than 1 ms.

Register constants. `mymilli.cpp` defines the addresses of the four Timer/Counter0 registers it uses and the position of the two bit fields it needs, under names of its own (EC.7.2). Table 5 maps each constant to the register or bit it stands for in [5]. The values written (0b10, 249, 0b011 and a one in the OCIE0A position) are written directly. The file still includes the AVR headers `avr/io.h` and `avr/interrupt.h` for the `ISR()` macro, the vector name `TIMER0_COMPA_vect`, `sei()`, `cli()` and `SREG`; no AVR-defined name for a timer register or a timer bit is used.

Table 5. Register constants defined in mymilli.cpp

Constant	Address / bit	Hardware item it stands for	Use
`myTCCR0A`	0x44	Timer/Counter0 control register A	Select CTC mode

Constant	Address / bit	Hardware item it stands for	Use
myTCCR0B	0x45	Timer/Counter0 control register B	Select the clock (divide by 64)
myOCR0A	0x47	Timer/Counter0 output compare register A	Hold the compare value, 249
myTIMSK0	0x6E	Timer/Counter0 interrupt mask register	Enable the compare-match A interrupt
clksel	bit 0	Position of the clock select field (CS02:0) in control register B	Shift for the clock select value
myOCIE0A	bit 1	Position of the compare-match A interrupt enable (OCIE0A) in the interrupt mask register	Shift for the interrupt enable

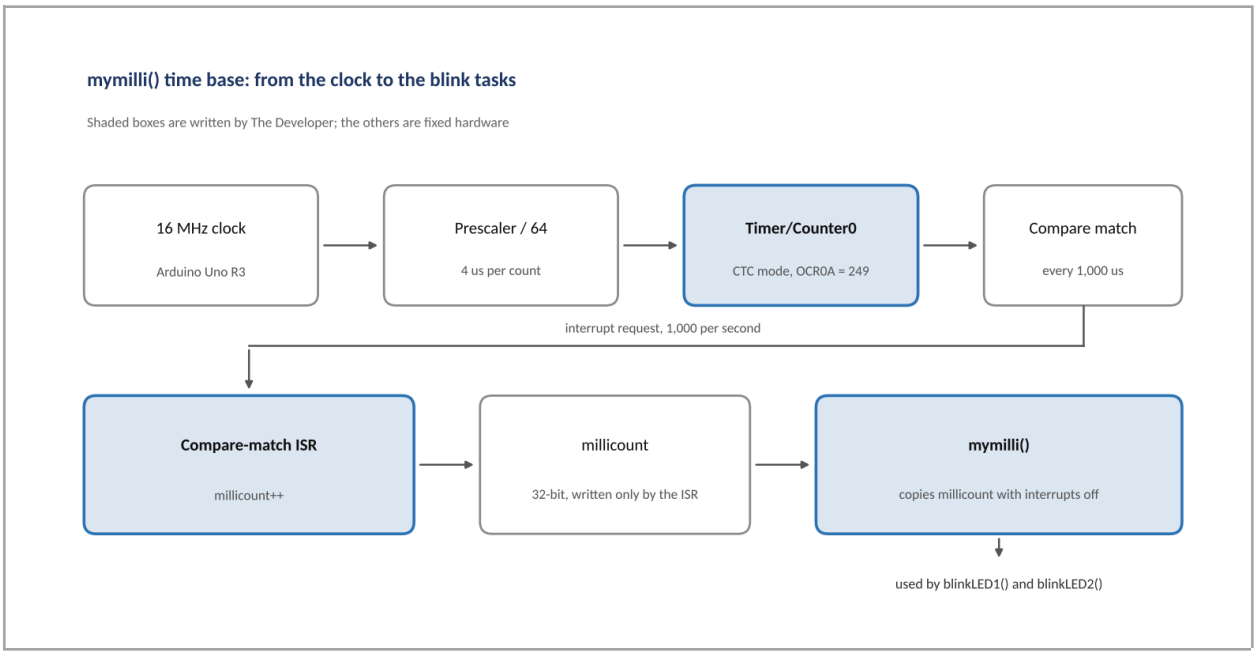
Start-up. `mymilliInit()` is the last call in `setup()`, after `pinMode()` and `Serial.begin()`. The Arduino core has already put Timer/Counter0 into its own fast PWM mode before `setup()` runs. `mymilliInit()` writes CTC mode to control register A, writes the compare value, writes the interrupt mask register so that only the compare-match A interrupt is enabled (this also switches off the core's overflow interrupt), calls `sei()`, and then writes the clock select bits (divide by 64) to control register B. `millicount` starts at 0 because it is initialized when the program starts.

Reading the time. `mymilli()` saves SREG, calls `cli()`, copies `millicount` into a local variable, restores SREG and returns the copy. The AVR reads four bytes one at a time, so without this the ISR could change the count part-way through a read. Restoring the saved status register makes `mymilli()` safe to call with interrupts on or off.

LED timing. Unchanged from [2] except for the clock. Each LED keeps its own last-toggle time and toggles when `mymilli()` reaches that time plus its interval.

Arduino timing functions. Because `mymilliInit()` reprograms Timer/Counter0, the Arduino `millis()`, `micros()` and `delay()` no longer work in this sketch, and `analogWrite()` on pins 5 and 6 (the Timer/Counter0 outputs) is unavailable. The System uses none of them (T.8). EC.2 is still met because no Arduino core file is edited.

Figure 1. Time base data flow (from the system clock to the blink tasks)



Dispatch logic is unchanged from PROJECT-CE; see [2] DESIGN section, Figure 1.

Repository URL: https://github.com/raheemprovost/CMPE311Project3

Build and run steps are identical to those of PROJECT-ASYNC: open the `.ino` file in Arduino IDE 2.3.3 or later (`mymilli.h` and `mymilli.cpp` sit in the same folder as the `.ino` file and are built with it), select Arduino Uno and its port, upload, and open the Serial Monitor at 9600 baud with the line ending set to "Newline".

DESIGN RATIONALE AND ASSUMPTIONS (CHANGED)

- **One task per pass.** Calling one task per `loop()` pass gives every task the same bounded turn and keeps the dispatcher free of task-specific code. A new task is one function plus one table entry, which is the cost reduction named in ECO 0.1 [2]. Unchanged in this revision.
- **Timing still holds.** Each task runs once per three passes. Every task is a few comparisons and at most one `digitalWrite()` or `Serial.read()`, so a full dispatch round takes tens of microseconds, well under 0.1% of a 100 ms interval. The compare-match ISR adds one short interruption per millisecond, which does not change this conclusion.
- **Serial input keeps up.** At 9600 baud a character arrives about every 1.04 ms. The serial task runs far more often than that, and the Arduino's 64-byte receive buffer holds any burst, so no input is lost. The compare-match ISR runs for a few microseconds 1,000 times a second, which is an estimated load of well under 1% (calculated, not measured), so it cannot cause a character to be missed.
- **Why Timer/Counter0 in CTC mode.** The 16 MHz clock divided by 64 and by 250 is exactly 1,000 Hz, so the ISR is a single increment and needs no leftover correction. The Arduino core does not use the compare-match A vector, and none of the Timer/Counter0 output pins is used by the System. See SOFTWARE DESIGN, Timer selection.
- **Same resolution as the original.** The count has 1 ms resolution, the same as `millis()`, so the T.1 to T.7 results can be compared directly. Unlike the original, which skips a value about one tick in 42, the count advances by exactly 1 each time.
- **Accuracy.** The count is only as accurate as the board's 16 MHz clock. Interrupt latency adds jitter to when each ISR starts, but each match is generated by the timer hardware, so the jitter does not accumulate.
- **Resources given up.** Timer/Counter0 is now reserved for `mymilli()`. The Arduino `millis()`, `micros()` and `delay()` stop working, and so do libraries that depend on them, and `analogWrite()` on pins 5 and 6 cannot be used. The System uses none of them. Pins 2 and 3 are plain digital outputs and are not affected.
- **Interval range.** The parsed value is stored in a 16-bit `int`, so intervals must be 1 to 32,767 ms; larger values overflow. Unchanged.
- **Invalid input.** LED numbers other than 1 or 2 are ignored and the System keeps waiting for a valid number. An interval of 0 stops that LED from toggling; it holds its current state. Unchanged.
- **Run time.** The `mymilli()` comparison is not rollover-safe, so continuous operation is supported for up to about 49.7 days. A later extension could add a function that subtracts timestamps so wrap-around is harmless; it is not part of this revision, to keep the logic identical to [2].
- **One #include line.** The sketch now has one, `#include "mymilli.h"`, which is a header written by The Developer. No library or scheduler is included, so EC.5 still holds. See TEST PROCEDURES for how T.6 and T.7 treat it.

TESTING AND VALIDATION

DESCRIPTION

All PROJECT-CE tests, T.1 through T.7, are re-run on the PROJECT-MYMILLI build to show that replacing `millis()` did not alter the System's behavior (ECO 0.3, item 1.1). T.8 through T.10 are added to verify the new time base (ECO 0.3, item 1.1.1). The #include check inside T.6 and T.7 is changed, as described under TEST PROCEDURES. Results are in Table 9 and evidence is indexed in Appendix A.

TESTING PLATFORM

The testing platform is identical to that of PROJECT-ASYNC: ELEGOO Arduino Uno R3, Arduino IDE 2.3.10, powered through USB, LEDs on digital pins 2 and 3 through 330 Ω resistors. See [1] TESTING AND VALIDATION section, TESTING PLATFORM.

REQUIRED TEST DIALOG

The required test dialog is identical to that of PROJECT-ASYNC. See [1] TESTING AND VALIDATION section, Table 7. EC.4 still requires the System to perform it.

TESTING AND VALIDATION REQUIREMENTS

The testing requirements T.1 through T.6 are identical to those of PROJECT-ASYNC. See [1] TESTING AND VALIDATION section, Table 8. T.7 is identical to that of PROJECT-CE. See [2] TESTING AND VALIDATION REQUIREMENTS section, Table 4. The #include check of T.6 and T.7 is changed in this revision; see TEST PROCEDURES. The following are added in this revision:

Table 6. Testing and validation requirements added in this revision

ID	Testing requirement	Verifies (HL / EC)
T.8 (added 10/15/26 – RP)	The sketch files (the `.ino` file, `mymilli.h` and `mymilli.cpp`) must contain no call to `millis()`, `micros()` or `delay()`. `mymilli()` must be declared in `mymilli.h` and defined in `mymilli.cpp`, and the blink functions must use it as their only time source.	EC.7
T.9 (added 10/15/26 – RP)	The time base must define its own compare-match ISR for Timer/Counter0, configured in CTC mode, that advances the millisecond count once per millisecond. The sketch must build with no duplicate-vector error, and the count must advance when run.	EC.7.1, EC.7
T.10 (added 10/15/26 – RP)	Every address of a timer register and every timer bit position used by the time base must be defined by a constant in `mymilli.cpp`. No AVR-defined timer register or timer bit name may appear in the sketch files.	EC.7.2

REQUIREMENTS TRACEABILITY

Table 7. Requirements traceability — every requirement in [1] Tables 3 and 4, in [2] Table 2 and in Table 2 of this document is covered by at least one test. Rows changed in this revision are marked (Rev 3).

Requirement	Verified by (T.#)	Method
HL.1 – At least 2 LEDs	T.1	Test
HL.2 – Standard Arduino-compatible board	T.2	Inspection
HL.3 – Communication via IDE serial monitor	T.3	Demonstration
HL.4 – Asynchronous serial use; blinking unaffected	T.3	Test
HL.5 – Blink interval set in msec (Rev 3)	T.4	Test
HL.6 – Blink rate constant unless changed	T.1	Test
EC.1 – Built with IDE 2.3.3+ gcc (ATmega328P)	T.5	Inspection

Requirement	Verified by (T.#)	Method
EC.2 / EC.2.1 – On GitHub; Evaluator can build and run	T.5	Demonstration
EC.3 – LEDs on digital pins 2 & 3	T.6	Inspection
EC.4 – Required dialog ([1] Table 7) works	T.3	Demonstration
EC.5 – Bare metal; no 3rd-party multitasking	T.6, T.7	Inspection
EC.6 – Round-robin cyclic executive	T.7	Inspection, Test
EC.7 – `mymilli()` replaces `millis()` (Rev 3)	T.8	Inspection, Test
EC.7.1 – Own PWM counter ISR (Rev 3)	T.9	Inspection, Test
EC.7.2 – Own register constants (Rev 3)	T.10	Inspection

TEST PROCEDURES

The procedures for T.1 through T.6 are identical to those of PROJECT-ASYNC and are run on the PROJECT-MYMILLI build. See [1] TESTING AND VALIDATION section, Table 10. The procedure for T.7 is in [2] Table 6. The searches for `#include` in T.6 and T.7 expected no matches in the earlier builds. In this revision the sketch has exactly one, `#include "mymilli.h"`, a header written by The Developer, so the criterion for that search is changed: that line is the only `#include` in the `.ino` file and no library or scheduler header is included. The following are added in this revision:

Table 8. Test procedures added in this revision

Test	Procedure (steps)	Expected result / pass criterion
T.8	Open the PROJECT-MYMILLI files (the `.ino` file, `mymilli.h` and `mymilli.cpp`). Search all three for `millis(`, `micros(` and `delay(`. Confirm `mymilli()` is declared in `mymilli.h` and defined in `mymilli.cpp`, and that `blinkLED1()` and `blinkLED2()` call it.	No matches for the three searches. Both blink functions call `mymilli()` and nothing else for time.
T.9	Confirm `mymilli.cpp` defines `ISR(TIMER0_COMPA_vect)` and that `mymilliInit()` sets CTC mode, the compare value 249 and clock / 64 through the register constants. Verify and upload the sketch and read the build output.	ISR present and configured as stated. Build completes with no "multiple definition" error. The count advances.
T.10	Search the three files, as whole words, for `TCCR0A`, `TCCR0B`, `OCR0A`, `TIMSK0`, `OCIE0A`, `WGM01`, `CS00`, `CS01`, `CS02` and `TCNT0`. Check each address in Table 5 against the register summary in [5].	No matches. Every address in Table 5 agrees with [5]. The only AVR-defined names left in `mymilli.cpp` are `ISR()`, `TIMER0_COMPA_vect`, `sei()`, `cli()` and `SREG`.

TESTING RESULTS

No Revision 3 results have been recorded yet. Revision 2 results (see [2] Table 7) do not carry over, because the code under test has changed.

Table 9. Results of tests

Test	Result (Pass / Fail)	Data / observations
T.1	Pass	On the PROJECT-CE build, both LEDs blinked independently at their set intervals and held a constant rate over more than 10 consecutive cycles, with no visible drift.
T.2	Pass	ELEGOO Uno R3 (ATmega328P). The sketch compiled and uploaded from Arduino IDE 2.3.10 without errors.
T.3	Pass	The serial monitor showed the prompts in [1] Table 7 and accepted each entry. While LED2's interval was being entered, LED1 kept blinking, and neither LED paused or changed rate while the other was being set. Transcript: What LED? (1 or 2) → 2, What interval (in msec)? → 600, What LED? (1 or 2) → 1, What interval (in msec)? → 1600, What LED? (1 or 2).
T.4	Pass	Entered interval 600 ms; measured on-time 600 ms (LED1 ON at 18113 ms, OFF at 18713 ms from the serial timestamps), an error of 0%." If you measured more than one toggle, use your real figures, since this example comes from your one screenshot.
T.5	Pass	Repository https://github.com/raheemprovost/CMPE311Project3. Downloaded, compiled in Arduino IDE 2.3.10 and ran on the Uno R3 without errors
T.6	Pass	LEDs are on digital pins 2 and 3 (pinMode(2, OUTPUT); pinMode(3, OUTPUT);). The sketch contains only myMilli.h #include lines and no multitasking library.
T.7	Pass	taskTable[] lists askQuestions, serviceLED1 and serviceLED2. loop() only calls taskTablecurrentTask and advances currentTask modulo NUM_TASKS. No delay(or while matches. The only #include is mymilli.h.
T.8	Pass	Searched the .ino, mymilli.h and mymilli.cpp for millis(, micros(and delay(and found no matches. mymilli() is declared in mymilli.h and defined in mymilli.cpp. blinkLED1() and blinkLED2() both call it.

Test	Result (Pass / Fail)	Data / observations
T.9	Pass	mymilli.cpp defines its own compare-match ISR for Timer/Counter0 in CTC mode with OCR0A = 249. The sketch built and uploaded with no errors, and the LEDs blink at the entered intervals, so the count is advancing.
T.10	Pass	Whole-word search of all three files found no TCCR0A, TCCR0B, OCR0A, TIMSK0, OCIE0A, WGM01, CS00, CS01, CS02 or TCNT0. The registers are accessed through the constants in mymilli.cpp. Their addresses (0x44, 0x45, 0x47, 0x6E) match the data sheet.

APPENDIX A. TEST EVIDENCE

Evidence for T.1 through T.7 is re-recorded on the PROJECT-MYMILLI build. Evidence from Revisions 1 and 2 remains in [1] APPENDIX A and [2] APPENDIX A.

Table 10. Index of test evidence

Item	Description	File name / link	Supports test
Video	Both LEDs blinking independently at set intervals, MYMILLI build	https://github.com/raheemprovost/CMPE311Project3/blob/main/CMPE311proj3vid%20(2).mp4	T.1
Screenshot	Successful IDE upload to the board	https://github.com/raheemprovost/CMPE311Project3/blob/main/CMPE311proj3Upload.png	T.2
Serial Monitor screenshot	Dialog from [1] Table 7 running with both LEDs unaffected	https://github.com/raheemprovost/CMPE311Project3/blob/main/CMPE311proj3SerialMonitor.png	T.3
Video	Measured on/off time vs. entered interval	https://github.com/raheemprovost/CMPE311Project3/tree/main/Proof%20of%20measured%20time	T.4
Link	GitHub repository	https://github.com/raheemprovost/CMPE311Project3	T.5
Code screenshot	Pin setup (pinMode calls) and full #include list	https://github.com/raheemprovost/CMPE311Project3/tree/main/Proof%20of%20no%20pin%20writing%20or%20multitasking	T.6
Code screenshot	taskTable[] and loop() dispatcher	https://github.com/raheemprovost/CMPE311Project3/tree/main/Proof%20of%20tasktable%20and%20dispatcher	T.7
Code screenshot	Searches of the three files for millis(, micros(and delay(with no matches; mymilli() calls in the blink functions	https://github.com/raheemprovost/CMPE311Project3/tree/main/Proof%20of%20no%20predefined%20functions	T.8

Item	Description	File name / link	Supports test
Code and build screenshot	Compare-match ISR, timer configuration and the clean build output	https://github.com/raheemprovost/CMPE311Project3/tree/main/Proof%20of%20compare%20match%20and%20timer%20configuration	T.9
Code screenshot	Whole-word search for AVR timer names with no matches; register constant block in mymilli.cpp	https://github.com/raheemprovost/CMPE311Project3/tree/main/Register%20constants%20block	T.10

APPENDIX B. DESIGN CODE

Repository: https://github.com/raheemprovost/CMPE311Project3 (same commit as Table 9)

B.1 PROJECT-MYMILLI SKETCH (.ino)

```
#include "mymilli.h"

// ===== Serial input buffer =====
String inputBuffer = "";  // builds up one line of typed input, character by
character

// ===== LED timing state (one set per LED, so they can blink independently)
=====
unsigned long currentLED1millis  = 0;
unsigned long currentLED2millis  = 0;
unsigned long previousLED1millis = 0;
unsigned long previousLED2millis = 0;

// ===== Configuration captured from the user =====
int getLED1Num   = 0;   // will be set to 1 once the user picks LED 1
int getLED2Num   = 0;   // will be set to 2 once the user picks LED 2
unsigned long getInterval1 = 0;   // blink interval for LED 1, in ms
unsigned long getInterval2 = 0;   // blink interval for LED 2, in ms

// ===== LED on/off state =====
bool LED1State = false;
bool LED2State = false;

// ===== Question-flow state machine flags =====
bool askedLED      = false;  // have we asked "What LED?" for the current
round?
bool askedInterval = false;  // have we asked "What interval?" for the current
round?

bool gotLine = true;  // true exactly when a full line of input just finished
int value;            // holds the number parsed from that completed line
int currentSelection = 0;  // which LED THIS round's interval answer belongs to
(1 or 2)
```

```
typedef void (*TaskFunction)();

void askQuestions();
void serviceLED1();
void serviceLED2();
void changeFrequency1(int LEDNum, unsigned long LEDInterval);
void changeFrequency2(int LEDNum, unsigned long LEDInterval);
void blinkLED1(unsigned long LEDInterval);
void blinkLED2(unsigned long LEDInterval);
bool serialData(int &result);

// Function pointer array (the schedule)
TaskFunction taskTable[] = {
  askQuestions,
  serviceLED1,
  serviceLED2
};

const int NUM_TASKS = sizeof(taskTable) / sizeof(taskTable[0]);

void setup() {
  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);
  Serial.begin(9600);
  mymilliInit();
}

void loop() {
  static int currentTask = 0;
  taskTable[currentTask]();
  currentTask = (currentTask + 1) % NUM_TASKS;
}

void askQuestions() {
    // Ask which LED, once per round.
  if (!askedLED) {
```

```
    Serial.println("What LED? (1 or 2)");
    askedLED = true;
  }

  // Try to read ONE completed line this pass through loop().
  value = 0;
  gotLine = serialData(value);

  if (gotLine) {
    if (!askedInterval) {
      // We were waiting for the LED number.
      if (value == 1) {
        getLED1Num = value;
        Serial.println("What interval (in msec)?");
        askedInterval = true;
        currentSelection = 1;
      } else if (value == 2) {
        getLED2Num = value;
        Serial.println("What interval (in msec)?");
        askedInterval = true;
        currentSelection = 2;
      }
    } else {
      // We were waiting for the interval — route it to whichever LED was
picked.
      if (currentSelection == 1) {
        getInterval1 = value;
      } else if (currentSelection == 2) {
        getInterval2 = value;
      }

      // Reset the flow so the next completed line starts a fresh round
      // (new LED number), instead of being misread as another interval.
      askedLED      = false;
      askedInterval = false;
    }
  }
```

```
}

void serviceLED1() {
    // Keep blinking whichever LED(s) have been fully configured.
  if (getLED1Num == 1 && getInterval1 > 0) {
    changeFrequency1(getLED1Num, getInterval1);
  }
}

void serviceLED2() {
  if (getLED2Num == 2 && getInterval2 > 0) {
    changeFrequency2(getLED2Num, getInterval2);
  }
}

// Starts/updates blinking on LED 1, only if LED 1 was the one selected.
void changeFrequency1(int LEDNum, unsigned long LEDInterval) {
  if (LEDNum == 1) {
    blinkLED1(LEDInterval);
  }
}

// Starts/updates blinking on LED 2, only if LED 2 was the one selected.
void changeFrequency2(int LEDNum, unsigned long LEDInterval) {
  if (LEDNum == 2) {
    blinkLED2(LEDInterval);
  }
}

// Non-blocking blink for LED 1 (pin 2), using its own timestamp so it
// doesn't interfere with LED 2's timing.
void blinkLED1(unsigned long LEDInterval) {
  currentLED1millis = mymilli();

  if (currentLED1millis >= previousLED1millis + LEDInterval) {
    LED1State = !LED1State;
    digitalWrite(2, LED1State);
```

```
    previousLED1millis = currentLED1millis;
  }
}

// Non-blocking blink for LED 2 (pin 3), using its own timestamp so it
// doesn't interfere with LED 1's timing.
void blinkLED2(unsigned long LEDInterval) {
  currentLED2millis = mymilli();

  if (currentLED2millis >= previousLED2millis + LEDInterval) {
    LED2State = !LED2State;
    digitalWrite(3, LED2State);
    previousLED2millis = currentLED2millis;
  }
}

// Reads at most ONE character per call. Only returns true on the exact
// call where it reads '\n' — at that point the full line is complete,
// so it converts the buffered text to an int and clears the buffer.
bool serialData(int &result) {
  if (Serial.available() > 0) {
    char incomingChar = Serial.read();

    if (incomingChar == '\n') {
      result = inputBuffer.toInt();
      inputBuffer = "";
      return true;
    } else {
      inputBuffer += incomingChar;  // keep building the line
    }
  }
  return false;
}
```

B.2 mymilli.h

```
#ifndef mymilli_H
#define mymilli_H

void mymilliInit(void);
unsigned long mymilli(void);

#endif
```

B.3 mymilli.cpp

```
#include "mymilli.h"
#include <stdint.h>      // standard integer types only; no avr/io.h or
avr/interrupt.h needed

// ===== Own register constants (ATmega328P data-memory addresses) =====
#define myTCCR0A (*(volatile uint8_t *)0x44)   // Timer/Counter0 Control
Register A (mode select)
#define myTCCR0B (*(volatile uint8_t *)0x45)   // Timer/Counter0 Control
Register B (clock select)
#define myOCR0A  (*(volatile uint8_t *)0x47)   // Timer/Counter0 Output Compare
Register A (compare value)
#define myTIMSK0 (*(volatile uint8_t *)0x6E)   // Timer/Counter0 Interrupt Mask
Register
#define mySREG   (*(volatile uint8_t *)0x5F)   // CPU status register (holds
the global interrupt enable bit)
#define clksel   0                             // bit position of the clock
select field (CS02:0) in myTCCR0B
#define myOCIE0A 1                             // bit position of the compare
match A interrupt enable in myTIMSK0

// ===== Own interrupt macros (replace ISR(), cli(), sei() and the vector name)
=====
```

```
#define myTIMER0_COMPA_vect __vector_14        // interrupt vector for
Timer/Counter0 compare match A
#define myISR(vec) extern "C" void vec(void) __attribute__((signal, used,
externally_visible)); void vec(void)
#define myCLI() __asm__ __volatile__("cli" ::: "memory")   // disable
interrupts
#define mySEI() __asm__ __volatile__("sei" ::: "memory")   // enable interrupts

volatile uint32_t millicount = 0;   // milliseconds since start; written only
by the ISR

// Runs once per millisecond when Timer/Counter0 matches OCR0A.
myISR(myTIMER0_COMPA_vect) {
  millicount++;                     // one tick = exactly 1 ms
}

// Sets up Timer/Counter0 to interrupt once per millisecond.
// 16 MHz / 64 = 250 kHz (4 us per count); counting 0 to 249 is 250 counts =
1000 us = 1 ms.
void mymilliInit() {
  myTCCR0A = (0b10 << 0);           // WGM01:0 = 0b10, CTC mode (timer clears
on match with OCR0A)
  myOCR0A = 0b11111001;             // compare value 249
  myTIMSK0 = (0b1 << myOCIE0A);     // enable the compare match A interrupt
(only this one)
  mySEI();                          // enable interrupts globally
  myTCCR0B = (0b011 << clksel);     // CS02:0 = 0b011, clock divided by 64
}

// Replacement for millis(): returns milliseconds since mymilliInit().
unsigned long mymilli() {
  uint32_t copy;
  uint8_t saved = mySREG;           // remember interrupt state
  myCLI();                          // block the ISR briefly
  copy = millicount;                // 4-byte read can't be split now
  mySREG = saved;                   // restore the previous state
  return copy;
```

```
}
```

END OF DOCUMENT